

MAS TRM Guidelines Compliance Analysis — Apache Fineract

Assessment of application-level controls against MAS Technology Risk Management Guidelines (January 2021)

March 9, 2026

Prepared for informational purposes. This document does not constitute legal or regulatory advice. Organisations deploying Apache Fineract should engage qualified professionals for a formal MAS TRM compliance assessment.

Table of Contents

1. Executive Summary	3
2. Access Control (TRM Section 9)	3
3. Authentication (TRM Section 9)	3
4. Audit Trail (TRM Section 11)	3
5. Cryptography / Data Protection (TRM Section 10)	4
6. Application Security (TRM Sections 6/7)	4
7. Multi-Tenancy & Data Isolation (TRM Section 10)	4
8. Notable Gaps Relative to Full MAS TRM Compliance	4
9. Conclusion	5

1. Executive Summary

Apache Fineract is an open-source core banking platform maintained under the Apache Software Foundation. While Fineract does not explicitly reference or target MAS TRM compliance, it implements several security controls that align with various sections of the MAS Technology Risk Management Guidelines (January 2021).

Full MAS TRM compliance requires organizational, operational, and infrastructure-level controls that extend well beyond the application layer. This document maps Fineract's application-level controls to the relevant TRM sections and identifies notable gaps.

2. Access Control (TRM Section 9)

- **Role-Based Access Control (RBAC):** Fineract implements granular RBAC through Role, Permission, and AppUser entities. Each API operation is guarded by permission checks at the service layer. Key classes include `fineract-core/.../useradministration/domain/Role.java` and `AppUser.java`.
- **URL-level Authorization:** `SecurityConfig` enforces URL-level authorization rules via Spring Security filter chains (`fineract-provider/.../infrastructure/core/config/SecurityConfig.java`).
- **Maker-Checker (4-Eye Principle):** Dual authorization is implemented via `PortfolioCommandSourceWritePlatformServiceImpl` and `CommandSourceService`. The system prevents the same user from acting as both maker and checker for the same command, enforcing separation of duties.
- **Data Scope (Office Hierarchy):** Access is restricted based on the office hierarchy. `SpringSecurityPlatformSecurityContext.validateAccessRights` ensures users can only access data within their organizational scope.

3. Authentication (TRM Section 9)

- **Multi-Factor Authentication:** Fineract supports HTTP Basic Auth, OAuth2 (JWT), and optional two-factor authentication via SMS or email OTP. The 2FA implementation is in `TwoFactorServiceImpl`.
- **Password Policies:** Configurable password complexity is enforced via `PasswordValidationPolicy`. The platform supports forced password reset on first login, password expiration, and prevention of password reuse.
- **Stateless Sessions:** Spring Security is configured with `SessionCreationPolicy.STATELESS`, which prevents session fixation attacks by ensuring no server-side HTTP sessions are created.

4. Audit Trail (TRM Section 11)

Fineract maintains a comprehensive audit trail for every non-read API request:

- All mutating commands are persisted to the `m_portfolio_command_source` table, recording the maker, checker, timestamps, client IP address, full command JSON, and processing status.
- Audit records are exposed via the `/v1/audits` REST API (`AuditsApiResource`), enabling external systems and compliance officers to query the trail.

- Resilience4j retry mechanisms are used to ensure reliable audit persistence even under transient failures.
- IP tracking is performed by CallerIpTrackingFilter, and correlation IDs support distributed tracing across service boundaries.

5. Cryptography / Data Protection (TRM Section 10)

- **AES-256 Encryption:** Tenant database passwords are encrypted using AES-256 with PBKDF2 key derivation, implemented in EncryptionUtil.
- **Password Hashing:** User passwords are hashed using bcrypt via Spring Security's DelegatingPasswordEncoder, providing strong one-way hashing with salting.
- **TLS / HSTS:** Transport Layer Security and HTTP Strict Transport Security are configurable in SecurityConfig, supporting encrypted communication channels.

6. Application Security (TRM Sections 6/7)

- **SQL Injection Prevention:** A multi-layered validation framework detects configurable attack patterns including blind injection, timing attacks, and stacked queries. Parameterized queries are enforced via the SQLBuilder utility class.
- **Content Upload Whitelisting:** File uploads are restricted by configurable regex patterns and MIME type validation, preventing malicious file uploads.
- **Idempotency:** The IdempotencyStoreFilter prevents duplicate request processing, ensuring each command is executed exactly once.
- **CORS:** Fully configurable Cross-Origin Resource Sharing policies allow organisations to restrict API access to authorized origins.

7. Multi-Tenancy & Data Isolation (TRM Section 10)

Fineract achieves strong data isolation in multi-tenant deployments through the following mechanisms:

- Each tenant is assigned a separate database schema, routed at runtime via RoutingDataSource. This ensures that one tenant's data is never accessible to another.
- Isolated connection pools with encrypted credentials are managed by DataSourcePerTenantServiceFactory, preventing credential leakage across tenants.

8. Notable Gaps Relative to Full MAS TRM Compliance

The following table summarises areas where Fineract's application-layer controls alone do not satisfy the full scope of MAS TRM requirements. Organisations must address these gaps through complementary governance, infrastructure, and operational measures.

TRM Area	Gap Description
IT Governance & Risk Management	No board-level risk framework
IT Project Management	No SDLC governance controls in codebase

Business Continuity / Disaster Recovery	No built-in DR orchestration
Data Loss Prevention	No DLP mechanisms at app layer
Intrusion Detection / Security Monitoring	No SIEM integration
Penetration Testing	Operational concern, not in codebase
Vendor / Outsourcing Risk	Not applicable at app level
Account Lockout	No account lockout after failed logins found
Session Timeout	Stateless API avoids sessions but no idle timeout for OAuth2 tokens
Data Masking / PII Protection	Limited evidence of data masking

9. Conclusion

Apache Fineract provides strong foundational controls for access control, audit logging, maker-checker dual authorization, encryption, two-factor authentication, password management, SQL injection prevention, and multi-tenant data isolation. These controls align well with several sections of the MAS TRM Guidelines.

However, full MAS TRM compliance requires complementary organizational governance, infrastructure hardening, operational procedures, and third-party security tooling. Financial institutions deploying Fineract should conduct a comprehensive gap analysis covering IT governance, business continuity, data loss prevention, intrusion detection, penetration testing, and vendor risk management to achieve full regulatory compliance.